

AUTONOMOUS MOBILE ROBOT (AMR) NAVIGATION USING PYTHON

Abstract

Autonomous Mobile Robots (AMRs) are intelligent robots capable of navigating environments without human intervention. This project demonstrates the implementation of a mobile robot that can detect obstacles, map its surroundings, and navigate from a start point to a destination using path-planning algorithms. The system uses simulated LiDAR data, occupancy grid mapping, and the A* algorithm for route planning. The project provides an introduction to autonomous navigation concepts used in modern robotics and industrial automation.

Introduction

Autonomous navigation is an important field in robotics that enables robots to move safely and efficiently in unknown environments. Mobile robots use sensors such as LiDAR, cameras, and ultrasonic sensors to perceive their surroundings. In this project, a simulated robot navigates through a maze-like environment while avoiding obstacles and finding the shortest path to its target.

Objectives

- To simulate an autonomous mobile robot.
- To create an occupancy grid map of the environment.
- To implement obstacle detection using simulated LiDAR.
- To find the shortest route using the A* algorithm.
- To navigate safely while avoiding obstacles.

Software Requirements

- Python 3.x
- Visual Studio Code
- Pygame Library

Hardware Requirements

- Computer/Laptop
- Internet Connection

Technologies Used

- Python Programming
- Robotics Fundamentals
- LiDAR Simulation
- Occupancy Grid Mapping
- A Path Planning Algorithm

Methodology

Step 1: Environment Creation

A two-dimensional grid map is created where:

- 0 represents free space.
- 1 represents obstacles or walls.

Step 2: Sensor Simulation

LiDAR sensing is simulated to detect nearby obstacles and estimate distances.

Step 3: Map Generation

The robot generates an occupancy grid map representing free and occupied spaces.

Step 4: Path Planning

The A algorithm calculates the shortest path from the starting position to the destination.

Step 5: Obstacle Avoidance

If an obstacle is detected on the planned route, the robot changes direction and continues navigation.

Algorithm

1. Initialize robot position.
2. Scan surroundings using simulated LiDAR.
3. Generate occupancy grid map.
4. Apply A* path planning.
5. Move toward destination.
6. Detect obstacles continuously.
7. Recalculate path if necessary.
8. Stop when destination is reached.

Results

The robot successfully:

- Detected obstacles.
- Generated a map of the environment.
- Found the shortest route using A*.
- Reached the destination without collisions.

Applications

- Warehouse Automation
- Autonomous Delivery Robots
- Industrial Robotics
- Smart Manufacturing
- Service Robots
- Hospital Transportation Systems

Advantages

- Reduces human effort.
- Improves navigation efficiency.
- Supports autonomous decision-making.
- Suitable for real-world robotic applications.

Limitations

- Accuracy depends on sensor quality.
- Complex environments require advanced SLAM techniques.
- Computational cost increases with map size.

Future Scope

- Integration with real LiDAR sensors.
- Implementation of SLAM algorithms.
- Addition of camera-based navigation.
- Deployment on physical mobile robots.

Conclusion

The Autonomous Mobile Robot Navigation project demonstrates the basic concepts of robotic perception, mapping, and path planning. Using occupancy grids and the A* algorithm, the robot successfully navigates through an environment while avoiding obstacles. This project serves as a strong foundation for learning advanced robotics and autonomous systems.

References

1. Python Documentation
2. Pygame Documentation
3. Robotics and Automation Fundamentals
4. A* Path Planning Algorithm Research Papers
5. Autonomous Mobile Robot Navigation Tutorials

```
File Edit Selection View Go Run Terminal Help
Restricted Mode is intended for safe code browsing. Trust this window to enable all features. Manage Learn More

robotic_arm.ipynb X occupancy_grid_maze.ipynb
C:\Users> Rohith > AppData > Local > Packages > 5319275A.WhatsAppDesktop.cv1g1gvanyjgm > LocalState > sessions > 79913AA0CD4496573F41D5A5C542E3F79FF3F54 > transfers > 2026-26 > robotic_arm.ipynb > import numpy as np
Generate + Code + Markdown Run All Select Kernel

import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation

# Arm lengths
L1 = 2
L2 = 2

# Target
x = 2
y = 2

# Inverse Kinematics
d = (x**2 + y**2 - L1**2 - L2**2)/(2*L1*L2)
theta2 = np.arccos(d)

theta1 = np.arctan2(y,x) - np.arctan2(
    L2*np.sin(theta2),
    L1 + L2*np.cos(theta2)
)

# Joint coordinates
x1 = L1*np.cos(theta1)
y1 = L1*np.sin(theta1)

x2 = x1 + L2*np.cos(theta1+theta2)
y2 = y1 + L2*np.sin(theta1+theta2)

# Plot
plt.plot([0,x1,x2],[0,y1,y2],
         marker='o',
         ...)
```

```
File Edit Selection View Go Run Terminal Help
Restricted Mode is intended for safe code browsing. Trust this window to enable all features. Manage Learn More

robotic_arm.ipynb X occupancy_grid_maze.ipynb
C:\Users> Rohith > AppData > Local > Packages > 5319275A.WhatsAppDesktop.cv1g1gvanyjgm > LocalState > sessions > 79913AA0CD4496573F41D5A5C542E3F79FF3F54 > transfers > 2026-26 > robotic_arm.ipynb > import numpy as np
Generate + Code + Markdown Run All Select Kernel

# Inverse Kinematics
d = (x**2 + y**2 - L1**2 - L2**2)/(2*L1*L2)
theta2 = np.arccos(d)

theta1 = np.arctan2(y,x) - np.arctan2(
    L2*np.sin(theta2),
    L1 + L2*np.cos(theta2)
)

# Joint coordinates
x1 = L1*np.cos(theta1)
y1 = L1*np.sin(theta1)

x2 = x1 + L2*np.cos(theta1+theta2)
y2 = y1 + L2*np.sin(theta1+theta2)

# Plot
plt.plot([0,x1,x2],[0,y1,y2],
         marker='o',
         linewidth=4)
plt.scatter(x2,y2,s=100)
plt.text(x2,y2,"End Effector")
plt.scatter(0,0,s=100)
plt.text(0,0,"Base")

plt.scatter(x1,y1,s=100)
plt.text(x1,y1,"Joint 1")
plt.xlim(-5,5)
plt.ylim(-5,5)
plt.grid()
plt.title("Robotic Arm Simulator")
plt.show()
```

